CODEBASE SECURITY & QUALITY AUDIT

SafetyMonitor 코드베이스 종합 Audit 보고서

산업현장 작업자 생체 모니터링 프로토타입 (IndusView / VitAlGuard)

작성일 2026-06-03 · **대상** monorepo @ `06.safetymonitor` · **방식** 9차원 멀티에이전트 스캔 + adversarial 검증
스코프 dashboard-web(Next.js) · backend/infrastructure(AWS CDK) · worker-phone(Flutter) · worker-watch-poc(watchOS) · packages · scripts
제외 node_modules · cdk.out · .next · build 산출물 · **원칙** 파일 무수정(보고 전용) · 모든 발견 file:line + 실제 코드 인용

1. 종합 결과 (Executive Summary)

9개 검사 차원을 독립 서브에이전트로 병렬 스캔하고, 각 발견사항을 adversarial 에이전트가 인용 코드를 재확인해 false positive를 제거했다. **확정 23건** (중복 1건 제거 후), false positive **10건+ 제거**.

Severity	건수	주요 항목
CRITICAL	0	—
HIGH	0	—
MEDIUM	5	PII 미인증 노출 · 기본 관리자 비밀번호 · 핵심 수신경로 테스트0 · tsconfig 플래그 누락 · 임계 로직 중복
LOW	18	보안1 · 정확성2 · 테스트2 · 의존성1 · dead code 6 · 문서4 · 타입2
합계	**23**	차원별 양호: **performance CLEAN** · analyzers 순수성 OK · 순환참조 0 · Swift force-unwrap 0

핵심 판정. CRITICAL/HIGH 0건은 결함이 없어서가 아니라 **프로토타입 스코프** 때문이다. 검증 단계에서 "실 서비스라면 HIGH"인 항목(PII 미인증 노출·기본 관리자 비밀번호)이 *세션마다 회전하는 임시 터널 + 합성/직원 데이터 + localhost 바인딩*이라는 실제 실행 경로 기준으로 MEDIUM으로 강등됐다. 즉 **발표 후 실서비스화의 첫 관문이 MEDIUM 5건에 모여 있다.**

가장 위험한 Top 5

#	제목	Severity	위치
1	미인증 GET `/api/mock/vital` 이 보호자 전화·GPS·이름 PII를 공개 터널로 유출	MED	`route.ts:12-19`
2	기본 관리자 자격증명 `admin/safety2026` 이 소스+.env.example에 박힘 → PII export 게이트	MED	`admin-auth.ts:5-7`
3	핵심 수신 게이트 `store.ts` 테스트 0건 (sticky merge 회귀 함정)	MED	`store.ts:29-119`
4	임계 분류 로직이 DemoDashboard에 매직넘버로 중복 → THRESHOLDS 단일 source 우회	MED	`DemoDashboard.tsx:160`
5	세션 토큰 = 가역 base64(id:password), 만료 없음 → 평문 비밀번호 영구 노출	LOW	`login/route.ts:39-41`

2. 취약점 (Security)

MEDIUM · **M1. 미인증 GET `/api/mock/vital` — 작업자 PII 평문 노출**

보안 · apps/dashboard-web/app/api/mock/vital/route.ts:12-19

```
export async function GET() {
  return NextResponse.json({
    latest: latestByWorker(),   // guardianName, guardianPhone, latitude, longitude 포함
    recent: recentSamples(),
    totalPersisted: totalPersisted(),
    countsByWorker: countsByWorker(),
  });
}
```

모든 `/api/admin/*` 라우트는 `verifyBasicAuth`를 거치지만 이 GET은 인증이 전혀 없다. 반환값에 **보호자 이름·전화, 위·경도, 작업자 이름**이 그대로 실린다(스키마 `vital-stream.ts:17-20`). dev 서버가 공개 `*.trycloudflare.com` 터널로 노출되므로 **URL만 알면 누구나 실시간 작업자 위치·보호자 연락처를 무인증 조회**할 수 있다.

MEDIUM 근거: 합성/직원 데이터 + 세션마다 회전하는 추측불가 임시 URL. 실서비스·고정 URL이면 HIGH.

보완 GET에 `verifyBasicAuth` 적용, 또는 공개 응답에서 PII 필드(전화·좌표·이름) strip 후 인증 요청에만 전체 제공. 최소 터널 공유 시크릿 헤더.

MEDIUM · **M2. 기본 관리자 자격증명 `admin / safety2026`** 보안 + 의존성 독립 2회 검출

보안·의존성 · apps/dashboard-web/lib/admin-auth.ts:5-7

```
const ADMIN_ID = process.env.ADMIN_ID ?? "admin";
// 환경변수 미설정 시 발표용 default. 실 서비스화 시 반드시 .env.local 로 override
const ADMIN_PASSWORD = process.env.ADMIN_PASSWORD ?? "safety2026";
```

`ADMIN_PASSWORD` 미설정 시 코드가 `"safety2026"`으로 fallback. **.env.example도 빈 placeholder가 아니라 실동작값을 그대로 출하**(같은 파일 `KOSHA_SERVICE_KEY=`는 비어 있는 것과 대조). 이 자격증명이 보호하는 `/api/admin/export`(CSV: `guardian_phone·heart_rate·spo2·latitude·longitude`)·`export-xlsx`·`workers`·`site-info`가 공개 터널에서 **리포지토리에 적힌 비밀번호로 접근 가능**.

MEDIUM 근거: 터널 활성 + env override 미설정이 동시 성립해야. 인증 자체는 `timingSafeEqual` 사용. 문서상 의도된 프로토타입 동작이나 노출은 실재.

보완 `.env.example`은 빈 placeholder로, 코드 fallback은 미설정 시 fail-closed(throw/인증거부). 소스에 리터럴 기본값 출하 금지.

LOW · **L3. 세션 토큰 = 가역 base64(id:password), 만료 없음**

보안 · apps/dashboard-web/app/api/admin/login/route.ts:39-41

```
const token = Buffer.from(`${id}:${password}`, "utf8").toString("base64");
return NextResponse.json({ ok: true, token });
```

토큰이 곧 평문 비밀번호이며 sessionStorage에 저장 후 매 요청 재생된다. XSS·확장프로그램이 읽으면 관리자 비밀번호가 영구 복원되고, 별도 폐기 가능 세션이 없다.

보완 서버측 TTL 매핑 opaque 세션 id(httpOnly 쿠키) 또는 단기 서명 JWT/HMAC.

3. 문제점 (정확성·테스트·타입·아키텍처)

MEDIUM · **M3. 핵심 수신 게이트 `store.ts` 테스트 0건**

테스트 · `apps/dashboard-web/app/api/mock/store.ts:29-50, 75-119`

```
export function addSample(input: unknown) {
  const normalized =
    typeof input === "object" && input !== null && "hr" in input
      ? { ...input, heartRate: (input as { hr?: unknown }).hr }
      : input;
  const sample = VitalSampleSchema.parse(normalized);
```

모든 vital 샘플의 단일 in-memory+DB 게이트인데 dashboard-web에 test 스크립트도 테스트 파일도 없다. 위험 표면: `hr→heartRate` 리맵, reserved 워커 필터, 500 ring slice, 특히 `latestByWorker`의 sticky merge(`sample.x ?? previous?.x`) — 주석이 명시 경고한 "`manualSos/fallDetected` 는 비-sticky" 규칙이 깨지면 전 작업자 카드가 조용히 오염되는데 잡을 테스트가 없다.

보완 node:test — 리맵·reserved null 반환·sticky 유지 + 이벤트 플래그 비-sticky 검증. `package.json`에 `test` 스크립트 추가.

MEDIUM · **M4. dashboard `tsconfig.json`이 base 미상속 → `noUncheckedIndexedAccess` 누락**

타입 안전성 · `apps/dashboard-web/tsconfig.json`

`tsconfig.base.json`은 `noUncheckedIndexedAccess: true`를 켜고 packages/analyzers·schemas는 이를 extends한다. 그러나 dashboard tsconfig는 `extends` 없이 옵션을 처음부터 정의해 `strict: true`만 있고 이 플래그가 빠졌다.
데모가 실제 렌더하는 가장 큰 런타임 표면에서 모든 배열 인덱스 접근이 타입 미검사가 된다.

예: `VitalHistoryPanel.tsx:366` `points[idx].label`, `typhoon.ts:185` `group[0]`이 definitely-present로 취급(오늘은 in-range지만 off-by-one/빈배열 회귀를 못 막음).

보완 dashboard tsconfig를 `"extends": "../../tsconfig.base.json"`로 하고 Next.js 옵션만 override(또는 `noUncheckedIndexedAccess: true` 명시).

MEDIUM · **M5. vital 위험도 분류 로직이 `DemoDashboard.tsx`에 매직넘버로 중복**

아키텍처 · `apps/dashboard-web/app/components/DemoDashboard.tsx:160-180`

```
function heartRateStatus(hr?: number): VitalStatus {
  if (hr === undefined) return "idle";
  if (hr >= 130 || hr <= 40) return "danger";   // 매직넘버 — THRESHOLDS 우회
  if (hr >= 110 || hr <= 50) return "caution";
  return "normal";
}
```

컴포넌트가 `heartRateStatus/spo2Status/coreTempStatus/overallStatus`를 인라인 재구현하며 HR/SpO₂ 경계를 생짜 숫자(130/40/110/50, 88/92)로 박았다. 이미 `lib/severity.ts:15-61`이 같은 분류를 THRESHOLDS(analyzers 단일 source) 참조로 수행하는데 컴포넌트는 temp만 THRESHOLDS를 쓰고 HR/SpO₂는 하드코딩. → `thresholds.ts` 임계 변경이 관제자가 실제 보는 카드 색칠 경로엔 전파되지 않는다. 동명 `overallStatus` 두 구현은 거동도 달라(severity.ts는 fall/SOS를 danger로 접지, 컴포넌트판은 누락) 유지보수 함정.

CLAUDE.md "analyzers THRESHOLDS 단일 source" 보장이 HR/SpO₂에서 실제로 깨져 있음

보완 컴포넌트의 4개 `*Status` 삭제 후 `lib/severity.ts`에서 import(camelCase↔snake_case 얇은 래퍼).

LOW · **L1. 워치 backlog 역순 수신 시 sticky `latestByWorker` 손상**

정확성 · `apps/dashboard-web/app/api/mock/store.ts:78-113`

`latestByWorker`가 삽입 순서 로 `ts: sample.ts`를 무조건 취한다. 워치 송신기는 fresh 배치 성공 후 *오프라인 backlog*(더 오래된 샘플)를 별도 POST로 보내므로(`VitalTransmitter.swift drainBacklog`) 옛 샘플이 나중 삽입 → ts 역행, 신선도 바가 stale 표시. 자가복구 (다음 샘플이 덮음)이며 SOS/낙상 경로는 별도 max-ts 가드라 안전.

보완 per-worker 최신을 max ts로 선택(또는 ts 역행 시 skip).

LOW · **L2. weather route가 `workers` 파라미터 부재 시 `workerCount=0` 영속**

정확성 · `apps/dashboard-web/app/api/weather/route.ts:92-97`

`Number(null) === 0`이 finite라 의도한 `undefined` 대신 `0`이 `weather_log`에 기록된다. 앱 경로(WeatherStrip은 항상 `&workers=N` 부착)는 미발동, 외부 직접호출만. 보완 raw 파라미터를 먼저 읽어 `null` 체크.

LOW · **L4. `VitalSampleSchema` 테스트 0 + test 스크립트가 no-op**

테스트 · `packages/schemas/src/vital-stream.ts:4-31`

모든 수신의 유일한 검증 게이트인데 `packages/schemas/package.json`의 `"test": "node --test ... src/*.test.ts"`가 매칭 파일 0개라 **조용히 통과**(false confidence). 보완 `vital-stream.test.ts` 추가(경계값 spo2 50, hr 20-250, datetime 등).

LOW · **L5. dashboard `severity.ts` 미테스트 (사용 경로)**

테스트 · `apps/dashboard-web/lib/severity.ts:15-61`

4개 admin export 라우트가 import하는 실제 severity 계산. caution 밴드(hr 41-50)는 `THRESHOLDS.lowBpm=40`과 달리 이 모듈 고유라 미검증 — `<` vs `<=` 회귀가 어디서도 안 잡힌다. 보완 경계값 테스트 + dashboard 테스트 러너.

LOW · **L17 / L18. 타입 우회 2건**

타입 안전성 · `db.ts:436` · `vital/route.ts:53`

L17 `persistSiteInfoDaily`가 `.get()` 결과(`Row|undefined`)를 비-nullable `SiteInfoRow`로 강제 캐스트(같은 파일 440행은 `| undefined`로 올바르게 처리). L18 `catch (error: any)`(TS 소스 유일 `: any`) → `catch (error: unknown)`로 변경(기존 `instanceof ZodError` 내로잉 그대로 동작).

Dead code (6건, LOW)

#	심볼 / 항목	위치	비고
L6	`SafetyMonitorLogo` 컴포넌트 (렌더 0)	`DemoDashboard.tsx:1655`	~26줄 미사용 SVG
L7	`SEVERITY_TEXT_CLASS` export (소비 0)	`weather-rules.ts:351`	형제 상수는 사용됨
L8	`persistedByWorker` DB 함수 (호출 0)	`db.ts:351`	store.countsByWorker로 대체됨
L9	`ADMIN_ID_HINT` export (사용 0)	`admin-auth.ts:40`	—
L10	`connectMqtt` 미호출 → MQTT 분기 도달불가	`vital_publisher.dart:46`	AWS 보류 의도 스캐폴딩 · mqtt_client 의존성도 이 경로용
L11	`WorkerSchema/HazardType/Severity` 외부 사용 0	`schemas/src/worker.ts:4-13`	미구현 백엔드용 forward 스캐폴딩

의존성 (1건, LOW)

> LOW · **L12. `geolocator_android/_apple` 직접 핀이 umbrella와 중복**
>
> 의존성 · `apps/worker-phone/pubspec.yaml:39-41`
>
> `geolocator`가 이미 endorsed federated로 끌어오고 `lib/` 직접 import 0. 현재 충돌은 없으나 향후 bump 시 마찰. 보완 두 platform 라인 제거.

문서·일관성 (4건, LOW)

#	내용	위치
L13	CLAUDE.md `# turbo run build/test` 주석 — turbo.json·의존성 부재, 실제는 `pnpm -r --filter`	`CLAUDE.md:205-206`
L14	`backend/README.md` 상태 stale(2026-05-27 · D2~75%/D3~55% · 만료 터널 · 마감 06-04) ↔ CLAUDE.md(D2~88%/D3~68% · 07-07). **현재 git M 표시 파일**	`README.md:3`
L15	README "4 worker" vs `mock-publish.mjs` 5 worker 불일치	`README.md:45`
L16	CLAUDE.md 코드구조 트리가 미존재 경로 나열(`dashboard-mobile`·`backend/lambdas/*`·`packages/thresholds`·`.github`) — "M1 진입 후 채움" 미래 트리	`CLAUDE.md:243-261`

4. 양호 항목 & 검증으로 제거된 오탐

✅ **performance — CLEAN (확정 0건).** 모든 `useEffect`의 setInterval/setTimeout/rAF/addEventListener/ResizeObserver가 cleanup에서 해제됨(useMarqueeScroll·DemoDashboard 2s폴/1s틱/테이블 rAF/제스처·RadarRealtime·TyphoonTrack 등 전수 확인). Flutter `main.dart _timer/_connSub`+컨트롤러 dispose, `background_keepalive.dart` StreamSubscription cancel. `db.ts`는 prepared statement 1회 생성 후 재사용(query-in-loop 없음). AudioContext statechange 리스너는 lifetime 싱글톤이라 누수 아님(재확인 후 drop).

✅ **그 외 양호.** packages/analyzers 순수성 OK(fetch/DB/AWS SDK/fs/process.env 없음) · 패키지 간 순환참조 0 · backend↔dashboard 계층 침범 0 · Swift force-unwrap/`try!`/`as!` 0 · Dart `!` 사이트 전부 `== null ||` 가드 · 외부 JSON 파서(typhoon/kma/kosha) 방어적 null-coalescing.

검증으로 제거된 대표 false positive (10건+)

오탐 후보	제거 사유
미인증 POST 무한 적재 (DB DoS)	임시 회전 터널 + 합성 데이터, 실 blast radius 미미
HR/SpO_2 "sustained" 단일샘플 발동	분석기 함수 **프로덕션 호출처 0** — 현재 미도달 경로(서버측 분석은 발표 후)
SOS 80-window eviction	실 수신율(1~2/s)로는 2s 폴 안에 SOS 샘플이 밀려날 수 없음
kma-warning (0,0) fallback	UI가 `undefined` 시 fetch 자체를 안 함 → 미도달
`pnpm test`가 dashboard 제외	dashboard에 테스트 자체가 없어 손실 0
mqtt_client 구조적 dead	의도된 AWS 로드맵 스캐폴딩(문서화됨), runtime 영향 0

5. 보완 권장 우선순위

순위	그룹	항목
A	**공개 터널 노출 전 (PII 차단)**	M1 GET PII strip/인증 · M2 `.env.local` 비번 설정+fail-closed · L3 세션토큰 분리
B	빠른 정확성 패치	L2 weather 가드(1줄) · L1 sticky ts 순서 가드
C	**단일 source 복구**	M5 DemoDashboard 임계 로직을 severity.ts로 통합 · M4 tsconfig base extends (둘 다 발표 전 권장 — 데모 렌더 경로 정확성 직결)
D	테스트 부채	M3+L4+L5 dashboard 테스트 러너+store/severity 테스트, schemas no-op 스크립트 실동작화
E	위생 (저위험 일괄)	L6-L11 dead code · L12 geolocator 핀 · L17/L18 타입 캐스트 · L13-L16 문서 갱신(특히 L14 현재 수정중 backend README)

6. 결론

CRITICAL/HIGH 0 — 프로토타입으로서 견고하다. 성능 누수 0, 분석기 순수성·계층 분리·null 안전성 모두 양호. 다만 "발표 후 실서비스화"의 첫 관문은 **MEDIUM 5건**(PII 인증 · 관리자 비밀번호 · 핵심 수신경로 테스트 · tsconfig 플래그 · 임계 단일 source)이며, 이 중 **M5(임계 로직 중복)는 지금도 "THRESHOLDS 단일 source" 문서 보장을 깨고 있어** 발표 전 정리를 권장한다. 공개 터널 시연을 한다면 그 직전 그룹 A(PII 차단) 3건이 가장 우선이다.